Try them all, or think ahead

Optimization · Level 3 · Lesson 1



In this lesson you will

- Count the loads a brute-force search must try.
- Write a brute-force knapsack search in Python.
- Explain how dynamic programming is sure without trying everything.

Key ideas

- $2^6 = 64$ loads; every item doubles it
- Greedy: 60; the best: 65
- Skip it, or take it and reuse the rest
- Whose points?

Activity: The best bag

1. Warm-up (3 min): Six things, weights and points.
2. Greedy (5 min): Most points per 100 g first.
3. Brute force (7 min): Run your program on your bag.
4. Compare (5 min): Did greedy miss the best?

No scale? Estimate: a bottle of water is about 500 g, a book about 300 g, a phone about 200 g.

Let’s discuss

1. Why does every extra item double the work for brute force?
2. When is a greedy answer good enough?
3. Who should choose the points in a real decision?

Exit ticket

- How many loads for 10 items + one example where greedy fails.